



Софийски университет „Св. Кл. Охридски“

Факултет по математика и информатика



Предмет: Системи за паралелна обработка

Летен семестър, 2025/2026 год.

Курсов проект

Паралелна имплементация на симулацията Wa-Tor

Изготвил:

Виктор Викторов Пъжев, фак. №: 4MI0800503

Ръководители:

проф. д-р Васил Цунижев

ас. Христо Христов

май, 2026

София

Съдържание

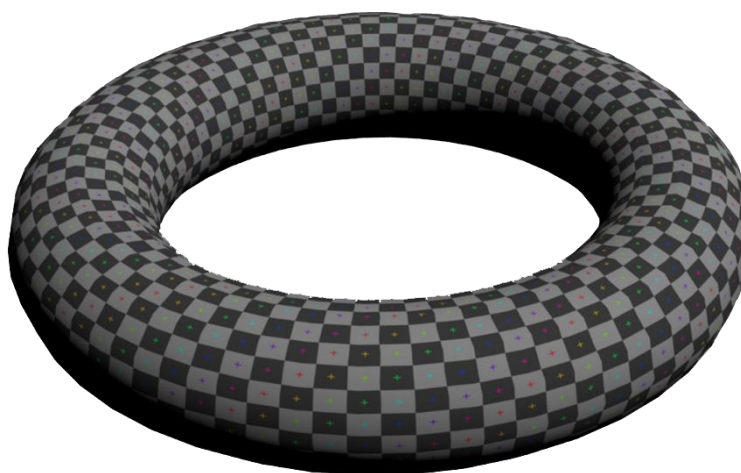
Въведение	3
Описание на симулацията	3
Анализ на задачата.....	4
Обзор на функционалностите в Wa-Tor и техните реализации	4
Използвани технологии	6
Реализация на задачата	7
Паралелен алгоритъм.....	7
Архитектура.....	8
Синхронизационни примитиви.....	11
Характеристики на тестовите машини.....	13
Тестови резултати.....	14
Визуализация	19
Заключение	23
Използвани източници.....	23

Въведение

Целта на настоящия проект е реализирането, оптимизирането и изследването на паралелен алгоритъм за симулиране на двумерен клетъчен автомат „Wa-Tor“ (модел хищник-жертва). Проектираното приложение има за цел да демонстрира максимизиране на ускорението и ефективността при разпределение на изчисленията между няколко нишки.

Описание на симулацията

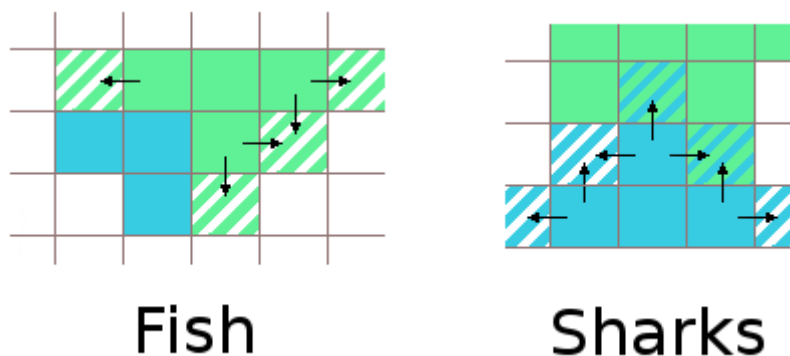
Симулацията Wa-Tor (произлиза от Water и Torus) моделира популационна динамика от тип „хищник-жертва“ и е въведена от Александър Дюдни през 1984 г. в списанието „Scientific American“ [1]. Светът представлява дискретна решетка върху тор: матрица, в която последният ред бива следван от първия, а последната колона – от първата, създавайки чувството за безкрайно циклично пространство.



ФИГУРА 1: СВЕТЪТ „WA-TOR“

Полетата биват 3 вида: вода, риби и акули. Времето в симулацията протича на дискретни стъпки, наречени хронони, като на всеки хронон рибите и акулите се местят и променят жизнените си показатели въз основа на строги правила:

- **Правила за рибите:** Всяка риба сканира четирите си ортогонални съседни клетки за свободно място и се премества в едно от тях на случаен принцип. Ако е оцеляла достатъчно дълго и е достигнала възраст за размножаване, при преместването си оставя нова риба на своята предишна позиция, а собствената ѝ възраст се нулира. Ако няма свободни съседни клетки, рибата остава на място.
- **Правила за акулите:** Акулите търсят съседна клетка, съдържаща риба. Ако открият такава, те се преместват върху нея и я изяждат, възстановявайки определено количество енергия. Ако няма съседни риби, акулата се премества в случайно избрана съседна празна клетка. На всеки изминал хронон акулата губи енергия и ако тя спадне до нула, хищникът умира от глад и се премахва от решетката. Аналогично на рибите, при достигане на възраст за размножаване, след преместване акулата създава нова акула на старата си позиция.



ФИГУРА 2: ДВИЖЕНИЕ НА РИБИТЕ И АКУЛИТЕ

Взаимодействието на тези елементи представлява симулация на диференциалните уравнения на Лотка-Волтера и след продължително изпълнение системата достига до едно от следните състояния:

1. Баланс между популациите на рибите и акулите;
2. Изчезване на акулите, остават само риби;
3. Изчезване на рибите и акулите.

Анализ на задачата

Обзор на функционалностите в Wa-Tor и техните реализации

Алгоритъмът за реализиране [1] представлява циклична обработка над двумерна матрица. Първоначално матрицата се запълва със случайно избрани (но с определени тежести) количества от риби, акули и вода. След това за всеки хронон матрицата се обхожда и за всяка клетка, която е риба или акула, се прилагат описаните горе правила. Броят операции расте с размера на матрицата и времето, поради което при големи стойности на широчината, височината и броя хронони, изпълнението на симулацията се забавя. Това води до нуждата от паралелна реализация, което значително ускорява изчисленията и може да се случи по различни начини.

Ще разгледаме 2 реализации на задачата, като ще ги характеризираме чрез 5 метрики:

1. **Максимален паралелизъм** – относително колко инструкции могат да се провеждат в един и същи момент.
2. **Синхронност** – определя модела на комуникация и времева координация между паралелните процеси. Алгоритъмът може да бъде локално-синхронен (синхронизация има между съседни процеси, които достъпват или обменят споделени гранични данни), глобално-синхронен или изцяло асинхронен.
3. **Декомпозиция** – стратегията за разделяне на проблема на части. Задачата може да се раздели по данни, при което матрицата се разделя на области като на редове, колони или квадратни сегменти, или по функции, при което нишките изпълняват различни специализирани роли.

4. **Балансиране** – начинът, по който изчислителното натоварване се разпределя между наличните нишки. То бива статично - задачите се разпределят фиксирано или циклично още при стартирането, или динамично - товарът се разпределя в реално време по време на изпълнението, например чрез централизиран диспечер.
5. **Грануларност** – колко подзадачи биват изпълнени от всяка нишка. При едра грануларност това са 1 – 10 подзадания, при средна са 10 – 32, а при фина повече.

Избраните за разглеждане паралелни реализации на Wa-Tor са следните:

1. **wator-sim от AgostinoRizzo [2]:** Това е реализация, която е изградена изцяло върху стандарта за обмен на съобщения, предназначен за клъстери и системи без споделена памет. Възможността за клъстерно изчисление увеличава паралелизма, отстранявайки ограниченията на система с един процесор. Проектът разглежда както едномерна декомпозиция по редове, така и двумерна блокова декомпозиция чрез дефиниране на виртуални декартови топологии. За да се реши проблемът със състезателния достъп, при предаването на граничните клетки по мрежата, алгоритъмът разпределя процесите шахматно на „алфа“ и „бета“ задачи. Този образец е силно мащабируем, но изисква изключително сложна асинхронна комуникация за поддържане на консистентността на решетката.
2. **ParallelWator от nicolotonci [3]:** Това е реализация, предназначена за локални мултипроцесорни системи, използваща библиотеката за нишки. Решетката е декомпозирана по данни чрез хоризонтално разделяне на редове със статично балансиране – всяка нишка получава равен брой редове. Синхронизацията се постига глобално чрез бариери в края на всеки хронон и локално чрез мютекси за предпазване от състезателни състояния, когато риби или акули пресичат границите на под-областите. Този проект е функционално по-близък до нашето решение.

Реализация	Макс. паралелизъм	Синхронност	Декомпозиция	Балансиране	Грануларност	Контекстни параметри
[2] wator-sim	Голям	Локално-синхронен	По данни	Статично	Едра	Масивни решетки
[3] ParallelWator	Среден	Локално-синхронен	По данни	Статично	Едра	Средни решетки
Текуща реализация	Среден	Локално-синхронен	По данни	Статично	Едра	Решетка 2000x2000, 1000 хронона

ТАБЛИЦА 1: СРАВНЕНИЕ МЕЖДУ ПАРАЛЕЛНИ РЕАЛИЗАЦИИ НА WA-TOR

Изборът на характеристиките за нашата реализация е пряко продиктуван от целевата хардуерна архитектура – локална система със споделена памет. Тъй като решението оперира в рамките на един компютър, максималният паралелизъм е ограничен до *среден* (спрямо броя хардуерни нишки на процесора). Естествена за текущата задача е декомпозицията по данни чрез нарязване на матрицата на едномерни хоризонтални ивици от по няколко реда. Тъй като първоначалното разпределение на съществата в Wa-Tor създава относително хомогенно изчислително натоварване, статичното балансиране се явява оптимално – то напълно елиминира

времето забавяне, което биха внесли динамичните опашки от задачи. За да се минимизира допълнително цената на синхронизацията и превключването на контекста, е избрана едра грануларност, при която всяка нишка получава масивен, непрекъснат блок от редове. Накрая, локално-синхронният модел гарантира висока производителност, тъй като нишките имат нужда да обменят информация и да се синхронизират чрез обща памет по граничните редове. Така се избягва тежката комуникация на асинхронните модели с обмен на съобщения.

Използвани технологии

Разгледаните реализации се базират на две различни хардуерни и софтуерни инфраструктури:

- **[2] wator-sim:** Изграден е на **C++ с MPI**. Изисква инфраструктура с разпределена памет (компютърен клъстер). Комуникацията се осъществява чрез обмен на съобщения. Този модел е много подходящ за масивни матрици, но усложнява алгоритъма и комуникацията със съобщения внася допълнително забавяне.
- **[3] ParallelWator:** Реализиран на **C с POSIX Threads (Pthreads)**. Изпълнява се върху локален мултипроцесор със споделена памет. Използва примитиви на операционната система (мютекси) за локална синхронизация на общите променливи.

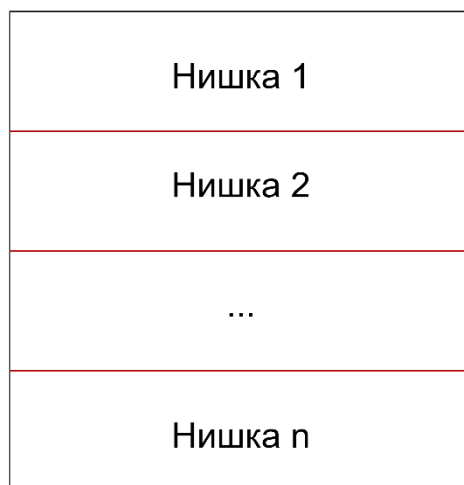
Текущият проект използва модел на локален мултипроцесор със споделена памет, тъй като мащабът на задачата не изисква разпределени клъстерни изчисления. За реализацията на алгоритъма е избран езикът **Java**, поради богатата вградена библиотека за конкурентно програмиране (`java.util.concurrent`):

- **Налични синхронизационни примитиви:** Java предоставя абстракции на високо ниво като `CyclicBarrier` (за фазите на хрононите) и `ReentrantLock` (за границите на под-областите). Това значително улеснява разработката, повишава четимостта на кода и минимизира риска от грешки, които са чести при ръчното управление на нишки в C/C++.
- **Фокус върху алгоритъма:** Използването на стабилна екосистема със споделена памет позволява фокусът на проекта да падне върху самия паралелен алгоритъм и декомпозицията на данните, вместо върху управлението на паметта на ниско ниво.

Реализация на задачата

Паралелен алгоритъм

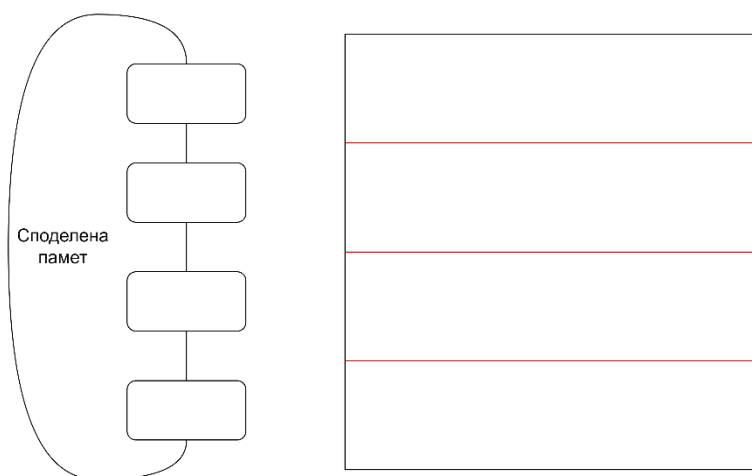
Представеният алгоритъм разделя матрицата на света Wa-Tor на ивици от по равен брой редове, всяка от които бива обработвана от отделна нишка, т.е. използвана е **декомпозиция по данни**. Тъй като невинаги броят редове може да се раздели поравно, последната нишка получава по-голяма ивица, включваща остатъка от целочисленото деление. Този остатък не влияе значително на нейната производителност, защото е от порядъка на броя подпроцеси, който е значително по-малък от общия брой редове (и съответно размера на всяка ивица). Всяка нишка изпълнява една и съща задача върху определената за нея област, което води до класифицирането на алгоритъма като **SPMD** (single program, multiple data).



ФИГУРА 3: РАЗДЕЛЕНИЕ НА ДАННИТЕ

Всяка нишка обработва точно една ивица, което значи, че броят на подпроцесите е равен на броя на подзадачите: **грануларността е едра** ($g = 1$). Разделянето на данните се случва преди създаването на нишките и зависи единствено от предварително зададения им брой от потребителя. Това води до заключението, че програмата работи на принципа на **статично балансиране**.

Отделните нишки не са изолирани напълно една от друга. Поради възможността риби и акули да преминават от една ивица на света в друга, се налага да се осъществи комуникация между нишките, отговарящи за съседни подобласти. Тъй като паралелните процеси работят върху една обща споделена матрица (но на различни нейни дялове), тази комуникация се осъществява именно чрез **споделена памет** (което налага внимателна синхронизация, вж. [„Синхронизационни примитиви“](#)). Връзката между съседните процеси създава впечатлението, че топологията на системата е линейна, но тороидалната структура на Wa-Tor налага да се използва **топология пръстен** (първата и последната ивица са съседни).



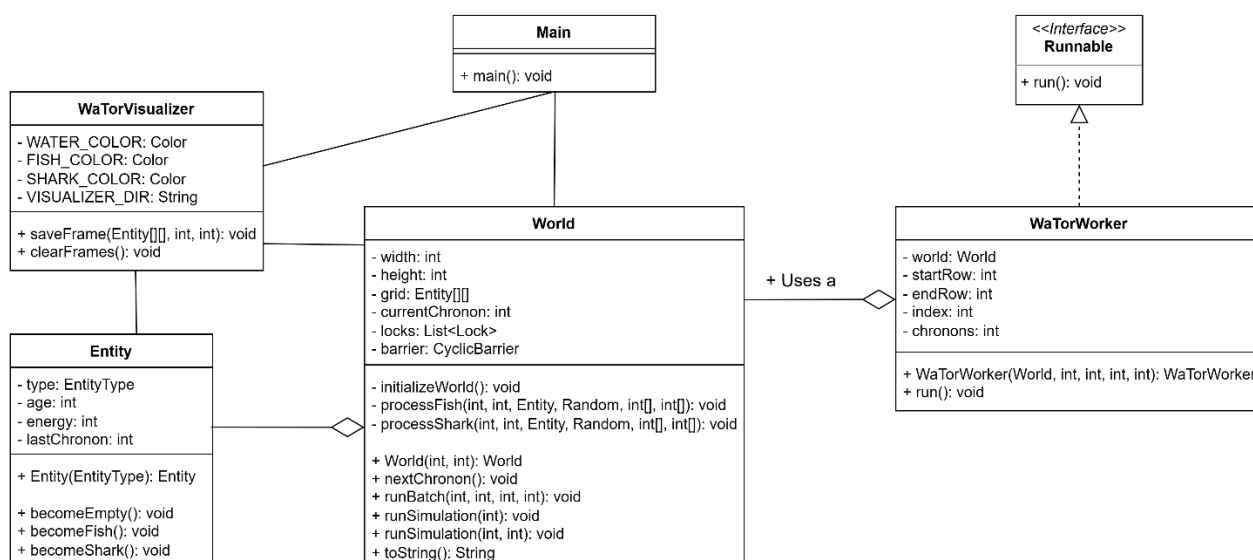
ФИГУРА 4: ТОПОЛОГИЯ НА ПРОЦЕСИТЕ

Архитектура

Имплементацията на алгоритма „Wa-Tor“ се състои от 4 Java класа:

- **Main** – отправната точка на приложението. Съдържа метода **main()**, чрез който потребителят въвежда размерите на матрицата, броя хронони и броя нишки и стартира изчислението.
- **World** – съдържа логиката, правилата и матрицата на симулацията. Конструкторът отговаря за генерирането на случаен свят, попълнен със случайно количество риби и акули (базирано на предварително дефинирано съотношение). Чрез **runSimulation()** се разпределят задачите между нишките и започва итерацията на хрононите. Методът **runBatch()** съдържа логиката, която изпълнява всеки подпроцес.
- **Entity** – моделира градивните частици на матрицата. Може да бъде вода, риба или акула и се характеризира с възраст, енергия и последен хронон, в който тази клетка е била обработена.
 - **EntityType** – енумерация, моделираща типа на клетката в матрицата. Може да бъде **EMPTY** (вода), **FISH** (риба) или **SHARK** (акула).
- **WaTorWorker** – имплементира интерфейса **Runnable** и абстрахира функционалността на отделна нишка. Съдържа информация за конкретната ивица от матрицата, за която отговаря, броя хронони, за които ще се изпълнява, и поредния си номер.
- **WaTorVisualizer** – отговаря за визуализирането на симулацията. Има статични методи за запазване на конкретно състояние на матрицата като картина и за изпразване на папката с картини.

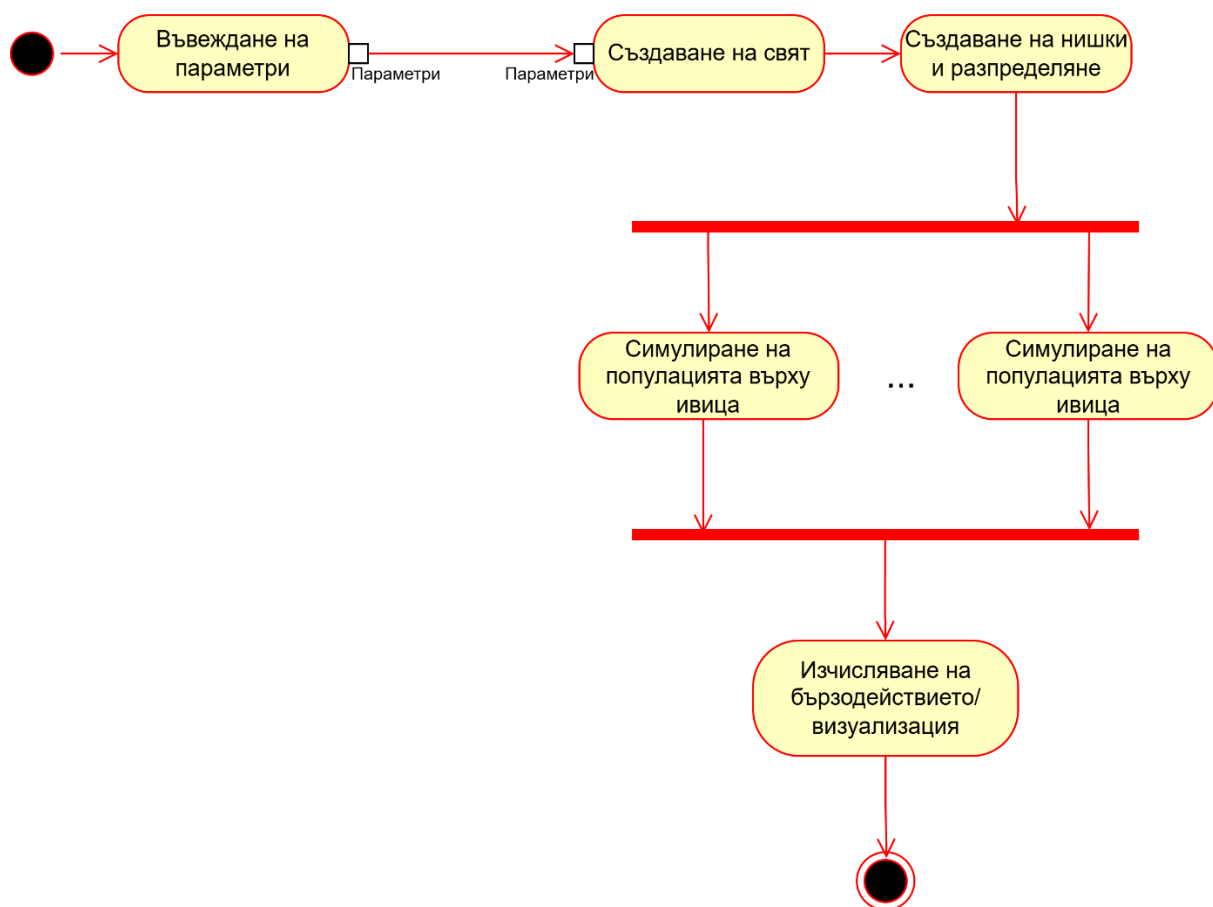
На следната UML клас диаграма е представена йерархичната зависимост между класовете:



ФИГУРА 5: UML КЛАС ДИАГРАМА

Класът **Main** използва класа **World**, който от своя страна използва класа **WaTorWorker** при създаване на нишките. **World** съдържа в себе си обекти от класа **Entity**, които изграждат матрицата на света. **WaTorWorker** съдържа в себе си референция към **World**, за да използва методите му. **WaTorVisualizer** бива използван в **Main** и **World** при избрана опция за визуализиране.

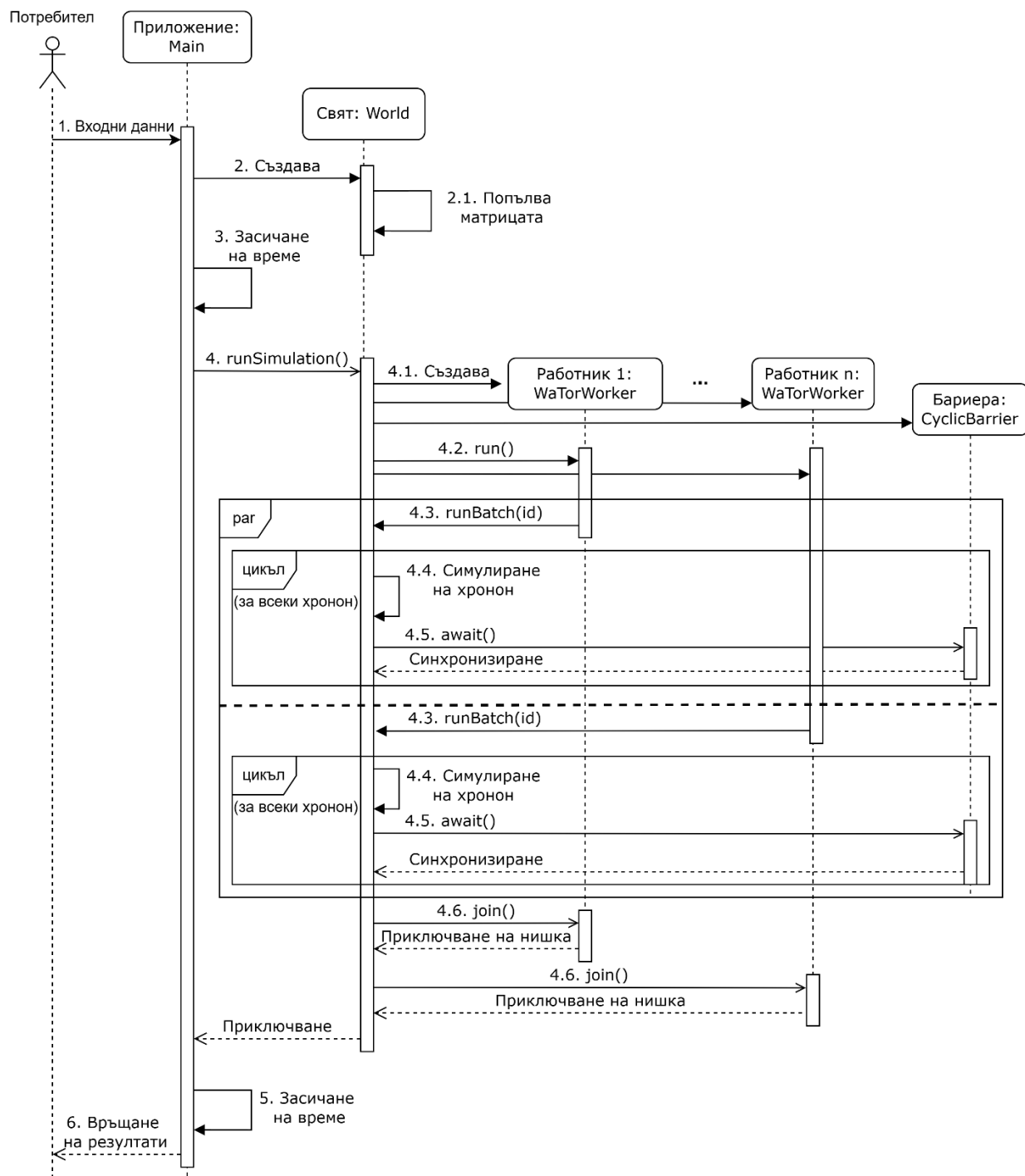
Следва диаграма на дейностите, илюстрираща процесите в приложението през времето:



ФИГУРА 6: UML ДИАГРАМА НА ДЕЙНОСТИТЕ

Процесът стартира последователно с въвеждане на конфигурационните параметри и инициране на обекта на симулационната среда, последвано от създаване и разпределяне на ивиците от редове към работните нишки. Чрез паралелно разклоняване се преминава от последователен към конкурентен режим на работа, при който отделните нишки едновременно изчисляват състоянието на популацията върху своите области от матрицата. Синхронизацията действа като бариера и гарантира изчакването на всички процеси, преди главната нишка да премине към финалното изчисляване на отминалото време или визуализацията на текущото състояние на света, след което симулацията приключва.

Следващата последователностна диаграма представя нагледно комуникацията на различните обекти в процеса на симулацията:



ФИГУРА 7: UML ПОСЛЕДОВАТЕЛНОСТНА ДИАГРАМА

Процесът започва с инициализация на симулационната среда (World) от главния клас, който стартира таймера за бързодействие и инициира изчисленията. World създава и стартира асинхронно работните нишки (WaTorWorker), които преминават в режим на паралелна обработка (обозначено чрез блока `par`). Всяка нишка самостоятелно извиква метода `runBatch()`, където започва свой локален цикъл за зададения брой хронони – изчислява новото състояние на поверената ѝ ивица от матрицата и в края на всяка итерация се синхронизира с останалите процеси посредством обща

циклична бариера (`CyclicBarrier.await()`), вж. „Синхронизационни примитиви“). След успешното приключване на всички итерации, нишките завършват изпълнението си и биват изчакани от главния поток чрез `join()`, след което системата финализира отчитането на времето и връща резултата на потребителя.

Синхронизационни примитиви

За реализиране на паралелната обработка се използва библиотеката на Java `java.util.concurrent`. Отделните нишки се инициализират чрез класа **Thread**, на който се подава обект от класа `WaTorWorker`, съдържащ кода за изпълнение. На Фигура 8 е показан програмният код, отговорен за паралелизацията:

```
public void runSimulation(int chronons, int threadsCount) { 1 usage  Viktor Pazhev
    currentChronon = 1;
    int batchSize = height / threadsCount;

    barrier = new CyclicBarrier(threadsCount, () -> ++currentChronon);

    Thread[] threads = new Thread[threadsCount];

    for (int i = 0; i < threadsCount; i++) {
        locks.add(new ReentrantLock());
    }

    for (int i = 0; i < threadsCount; i++) {
        threads[i] = new Thread(
            new WaTorWorker(
                world: this,
                startRow: i * batchSize,
                (i == threadsCount - 1) ? height - 1 : (i + 1) * batchSize - 1,
                i,
                chronons
            )
        );

        threads[i].start();
    }

    for (int i = 0; i < threadsCount; i++) {
        try {
            threads[i].join();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            System.out.println(e.getMessage());
        }
    }

    locks.clear();
}
```

ФИГУРА 8: ИМПЛЕМЕНТАЦИЯ НА ПАРАЛЕЛИЗМА

Акулите и рибите могат да преминават от областта от света, определена на една нишка, към друга такава. Това налага комуникация между съседните ивици, което се осъществява чрез споделена памет. Тази памет трябва да бъде защитена от състезателен достъп чрез ключалка. С тази цел е използван класът **ReentrantLock**, играещ ролята на мютекс. Споделената памет, която трябва да се защити, обхваща първите и последните два реда от всяка ивица. Вторият и предпоследният ред се включват, защото риби от тях имат възможността да се преместят в първия и последния ред на позиции, на които в същия момент може да се позиционира риба от съседната област. Това са общо p на брой групи от по два реда (p = брой нишки), за всяка от които се създава по една ключалка, събрани в лист. Когато даден подпроцес започне да обработва някои от тези групи, той първо заключва съответната част от кода, блокирайки други процеси от това, да манипулират същите данни. Когато обработката приключи, процесът освобождава съответната ключалка. Това взаимодействие е реализирано чрез следния програмен код:

```
for (int c = 0; c < chronons; ++c) {
    for (int y = startRow; y <= endRow; ++y) {
        if (y == startRow || y == startRow + 1) {
            locks.get(batchNum).lock();
        }

        if (y == endRow || y == endRow - 1) {
            locks.get((endRow == height - 1) ? 0 : (batchNum + 1)).lock();
        }

        try {...} finally {
            if (y == startRow || y == startRow + 1) {
                locks.get(batchNum).unlock();
            }

            if (y == endRow || y == endRow - 1) {
                locks.get((endRow == height - 1) ? 0 : (batchNum + 1)).unlock();
            }
        }
    }

    barrier.await();
}
```

ФИГУРА 9: ЗАЩИТА НА СПОДЕЛЕНА ПАМЕТ

За да се гарантира консистентността и коректното преместване на обектите през границите на ивиците, в алгоритъма е въведена глобална точка на синхронизация чрез обекта **CyclicBarrier**. Барьерата принуждава всяка работна нишка, приключила изчисленията за своя участък

през определен хронон, да изчака останалите, преди да продължи напред към следващия хронон. По този начин се предотвратяват логически аномалии – като преминаване на риба в територия, която е един хронон напред или назад във времето.

Въпреки че бариерата синхронизира всички нишки едновременно, самият архитектурен модел се класифицира като **локално-синхронен**. Главната нишка има само инициализираща роля, докато всяка работна нишка изпълнява свой собствен цикъл за симулационните стъпки. Вместо централен диспечер да събужда и приспива процесите за всеки отделен хронон (което би дефинирало глобално-синхронен модел), тук нишките се стартират само веднъж и сами управляват кога да извикат изчакващия примитив в края на локалната си итерация.

Характеристики на тестовите машини

За тестване и измерване на бързодействието и ускорението на алгоритъма бяха използвани две различни машини:

1. Домашен настолен компютър:

- Архитектура: x86_64
- Модел на процесора: AMD Ryzen 7 3700X 8-Core
- CPU(s): 16
- Брой нишки на ядро: 2
- L1d кеш: 32 KiB
- L1i кеш: 32 KiB
- L2 кеш: 512 KiB
- L3 кеш: 16 MiB
- RAM памет: 32 GB

2. Сървър във ФМИ (t5600.rmi.yaht.net):

- Архитектура: x86_64
- Модел на процесора: Intel(R) Xeon(R) CPU E5-2660 0 @ 2.20GHz
- CPU(s): 32
- Брой нишки на ядро: 2
- L1d кеш: 32 KiB
- L1i кеш: 32 KiB
- L2 кеш: 256 KiB
- L3 кеш: 20480 KiB
- RAM памет: 64 GB

Тестови резултати

Алгоритъмът беше тестван на 2 машини с по 2 сценария съответно за размер на света и брой хронони:

1. 1500 x 1500 матрица, 500 хронона;
2. 2000 x 2000 матрица, 500 хронона;
3. 1500 x 1500 матрица, 1000 хронона;
4. 2000 x 2000 матрица, 1000 хронона.

Други параметри на симулацията включват:

- Начален относителен брой акули: 10%
- Начален относителен брой риби: 30%
- Време за размножаване на рибите: 3
- Време за размножаване на акулите: 10
- Енергия, получена след изяждане на риба: 3
- Начална енергия на акулите: 5

Тестови метрики:

- p - брой нишки
- $T_p^{(i)}$ - време за изпълнение на i -ти тест с p нишки
- $T_p = \min(T_p^{(i)})$ – най-кратко време за изпълнение
- $S_p = T_1 / T_p$ - ускорение
- $E_p = S_p / p$ – ефективност (нормализирано ускорение)

Резултати от тестването на домашен компютър [милисекунди]:

#	p	$T_p^{(1)}$	$T_p^{(2)}$	$T_p^{(3)}$	$T_p = \min(T_p^{(i)})$	$S_p = T_1 / T_p$	$E_p = S_p / p$
1	1	33071	33790	36083	33071	1.0000	1.0000
2	2	18542	17036	16607	16607	1.9914	0.9957
3	4	9174	12496	9565	9174	3.6049	0.9012
4	8	6361	7140	6541	6361	5.1990	0.6499
5	12	5748	5279	5097	5097	6.4883	0.5407
6	16	5169	5446	4978	4978	6.6434	0.4152
7	20	6028	5851	5791	5791	5.7108	0.2855
8	24	5444	5489	5337	5337	6.1966	0.2582
9	28	5206	5416	4879	4879	6.7782	0.2421
10	32	5730	5286	5433	5286	6.2563	0.1955

ТАБЛИЦА 2: УСКОРЕНИЕ ПРИ 1500x1500, 500 ХРОНОНА, ДОМАШЕН КОМПЮТЪР

#	p	$T_p^{(1)}$	$T_p^{(2)}$	$T_p^{(3)}$	$T_p=\min(T_p^{(i)})$	$S_p = T_1/T_p$	$E_p = S_p/p$
1	1	64714	58523	61279	58523	1.0000	1.0000
2	2	34497	32793	30057	30057	1.9471	0.9735
3	4	16311	16490	16459	16311	3.5879	0.8970
4	8	10653	12126	12428	10653	5.4936	0.6867
5	12	10629	9995	9830	9830	5.9535	0.4961
6	16	9724	10319	9665	9665	6.0551	0.3784
7	20	10056	10309	9521	9521	6.1467	0.3073
8	24	9350	9951	9664	9350	6.2591	0.2608
9	28	9724	8853	9191	8853	6.6105	0.2361
10	32	9348	9676	9615	9348	6.2605	0.1956

ТАБЛИЦА 3: УСКОРЕНИЕ ПРИ 2000x2000, 500 ХРОНОНА, ДОМАШЕН КОМПЮТЪР

#	p	$T_p^{(1)}$	$T_p^{(2)}$	$T_p^{(3)}$	$T_p=\min(T_p^{(i)})$	$S_p = T_1/T_p$	$E_p = S_p/p$
1	1	70249	69094	69823	69094	1.0000	1.0000
2	2	36170	35703	35482	35482	1.9473	0.9736
3	4	19543	19080	20811	19080	3.6213	0.9053
4	8	13509	12956	13302	12956	5.3330	0.6666
5	12	11039	11691	10948	10948	6.3111	0.5259
6	16	10960	11423	10883	10883	6.3488	0.3968
7	20	11980	12319	11934	11934	5.7897	0.2895
8	24	11166	11002	11287	11002	6.2801	0.2617
9	28	10660	11139	10992	10660	6.4816	0.2315
10	32	12255	12035	11132	11132	6.2068	0.1940

ТАБЛИЦА 4: УСКОРЕНИЕ ПРИ 1500x1500, 1000 ХРОНОНА, ДОМАШЕН КОМПЮТЪР

#	p	$T_p^{(1)}$	$T_p^{(2)}$	$T_p^{(3)}$	$T_p=\min(T_p^{(i)})$	$S_p = T_1/T_p$	$E_p = \frac{S_p}{p}$
1	1	124617	125725	126458	124617	1.0000	1.0000
2	2	65111	65270	67220	65111	1.9139	0.9570
3	4	36197	37877	38795	36197	3.4427	0.8607
4	8	23418	23533	23706	23418	5.3214	0.6652
5	12	20686	21275	20779	20686	6.0242	0.5020
6	16	19590	20390	22092	19590	6.3613	0.3976
7	20	22946	21542	21682	21542	5.7848	0.2892
8	24	21064	21843	20744	20744	6.0074	0.2503
9	28	20316	20392	21269	20316	6.1339	0.2191
10	32	21577	21632	20081	20081	6.2057	0.1939

ТАБЛИЦА 5: УСКОРЕНИЕ ПРИ 2000x2000, 1000 ХРОНОНА, ДОМАШЕН КОМПЮТЪР

Резултати от тестването на сървъра t5600.rmi.yaht.net [милисекунди]:

#	p	$T_p^{(1)}$	$T_p^{(2)}$	$T_p^{(3)}$	$T_p = \min(T_p^{(i)})$	$S_p = T_1/T_p$	$E_p = S_p/p$
1	1	57726	57596	57724	57596	1.0000	1.0000
2	2	32639	29388	32070	29388	1.9598	0.9799
3	4	16691	16746	16977	16691	3.4507	0.8627
4	8	8897	9387	9241	8897	6.4736	0.8092
5	12	7056	6957	6862	6862	8.3935	0.6995
6	16	6582	6077	6257	6077	9.4777	0.5924
7	20	6613	6302	6563	6302	9.1393	0.4570
8	24	5904	5731	6045	5731	10.0499	0.4187
9	28	5417	5695	5829	5417	10.6325	0.3797
10	32	6520	6198	5918	5918	9.7323	0.3041

ТАБЛИЦА 6: УСКОРЕНИЕ ПРИ 1500x1500, 500 ХРОНОНА, СЪРВЪР ФМИ

#	p	$T_p^{(1)}$	$T_p^{(2)}$	$T_p^{(3)}$	$T_p = \min(T_p^{(i)})$	$S_p = T_1/T_p$	$E_p = S_p/p$
1	1	102918	102447	102655	102447	1.0000	1.0000
2	2	54144	56423	54480	54144	1.8921	0.9461
3	4	27297	27317	27814	27297	3.7530	0.9383
4	8	15994	16017	15293	15293	6.6989	0.8374
5	12	11358	11170	11618	11170	9.1716	0.7643
6	16	9797	9325	9116	9116	11.2382	0.7024
7	20	18886	11457	11718	11457	8.9419	0.4471
8	24	10754	14283	10269	10269	9.9763	0.4157
9	28	9700	11691	10066	9700	10.5615	0.3772
10	32	9934	9597	9588	9588	10.6849	0.3339

ТАБЛИЦА 7: УСКОРЕНИЕ ПРИ 2000x2000, 500 ХРОНОНА, СЪРВЪР ФМИ

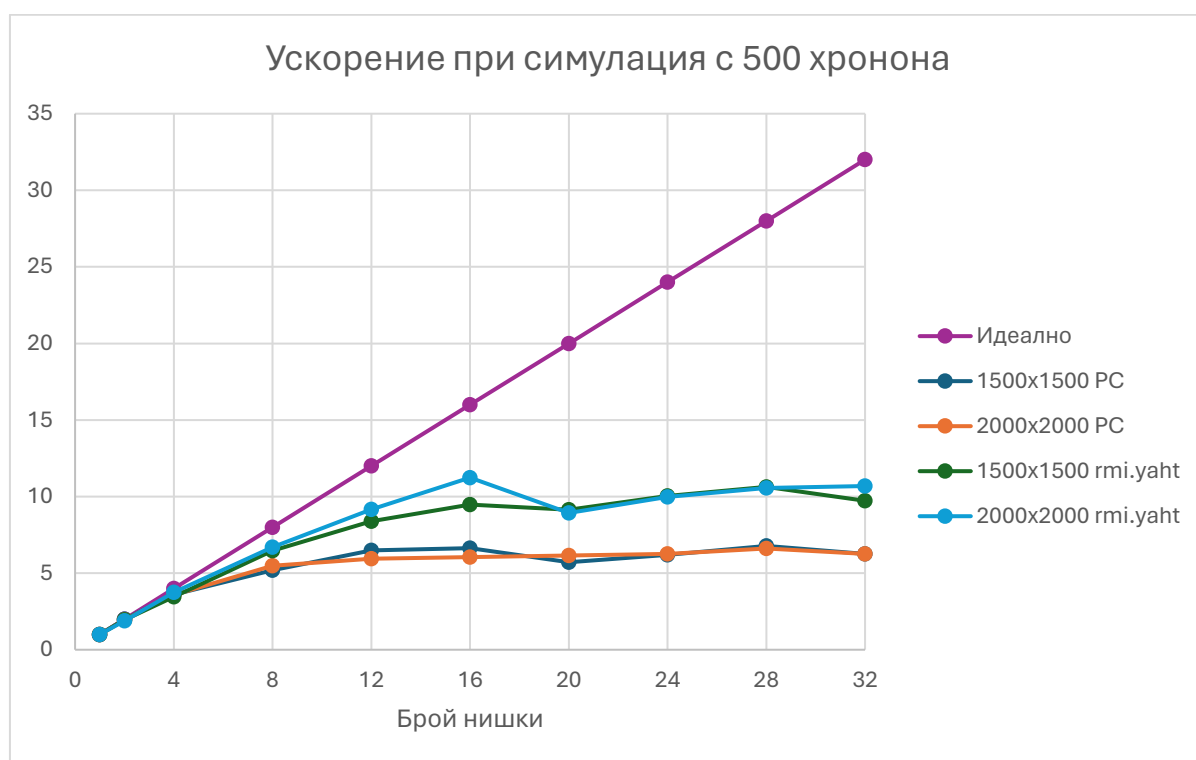
#	p	$T_p^{(1)}$	$T_p^{(2)}$	$T_p^{(3)}$	$T_p = \min(T_p^{(i)})$	$S_p = T_1/T_p$	$E_p = S_p/p$
1	1	119881	119423	119443	119423	1.0000	1.0000
2	2	65394	64112	64441	64112	1.8627	0.9314
3	4	32087	33683	33425	32087	3.7218	0.9305
4	8	18825	18910	18656	18656	6.4013	0.8002
5	12	13699	13408	13352	13352	8.9442	0.7454
6	16	12397	12352	12254	12254	9.7456	0.6091
7	20	12476	12680	13064	12476	9.5722	0.4786
8	24	11665	11797	11739	11665	10.2377	0.4266
9	28	10889	11246	10919	10889	10.9673	0.3917
10	32	11915	11626	11020	11020	10.8369	0.3387

ТАБЛИЦА 8: УСКОРЕНИЕ ПРИ 1500x1500, 1000 ХРОНОНА, СЪРВЪР ФМИ

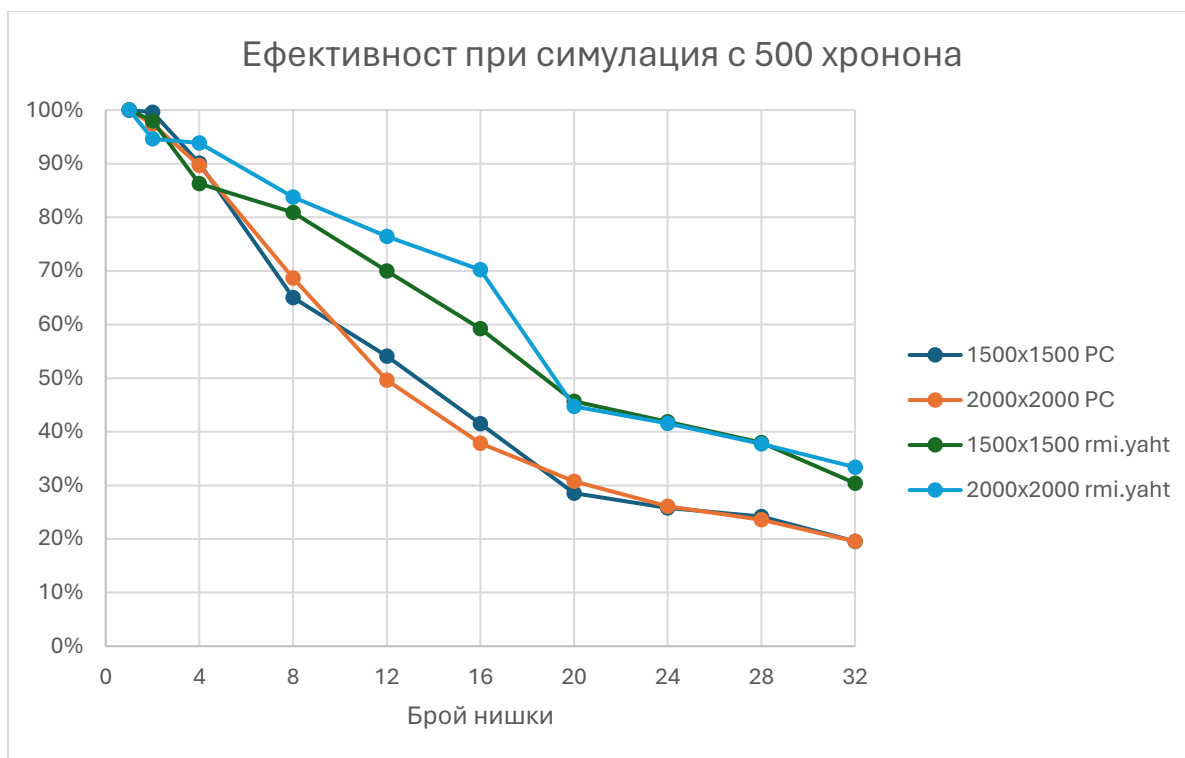
#	p	$T_p^{(1)}$	$T_p^{(2)}$	$T_p^{(3)}$	$T_p = \min(T_p^{(i)})$	$S_p = T_1/T_p$	$E_p = \frac{S_p}{p}$
1	1	213373	212486	214078	212486	1.0000	1.0000
2	2	117368	110990	112158	110990	1.9145	0.9572
3	4	59773	56814	59544	56814	3.7400	0.9350
4	8	33409	32947	33422	32947	6.4493	0.8062
5	12	24426	23254	23286	23254	9.1376	0.7615
6	16	20508	21684	20373	20373	10.4298	0.6519
7	20	23064	24467	28223	23064	9.2129	0.4606
8	24	22536	22022	32892	22022	9.6488	0.4020
9	28	21933	19008	20686	19008	11.1788	0.3992
10	32	20914	18529	19032	18529	11.4678	0.3584

ТАБЛИЦА 9: УСКОРЕНИЕ ПРИ 2000x2000, 1000 ХРОНОНА, СЪРВЪР ФМИ

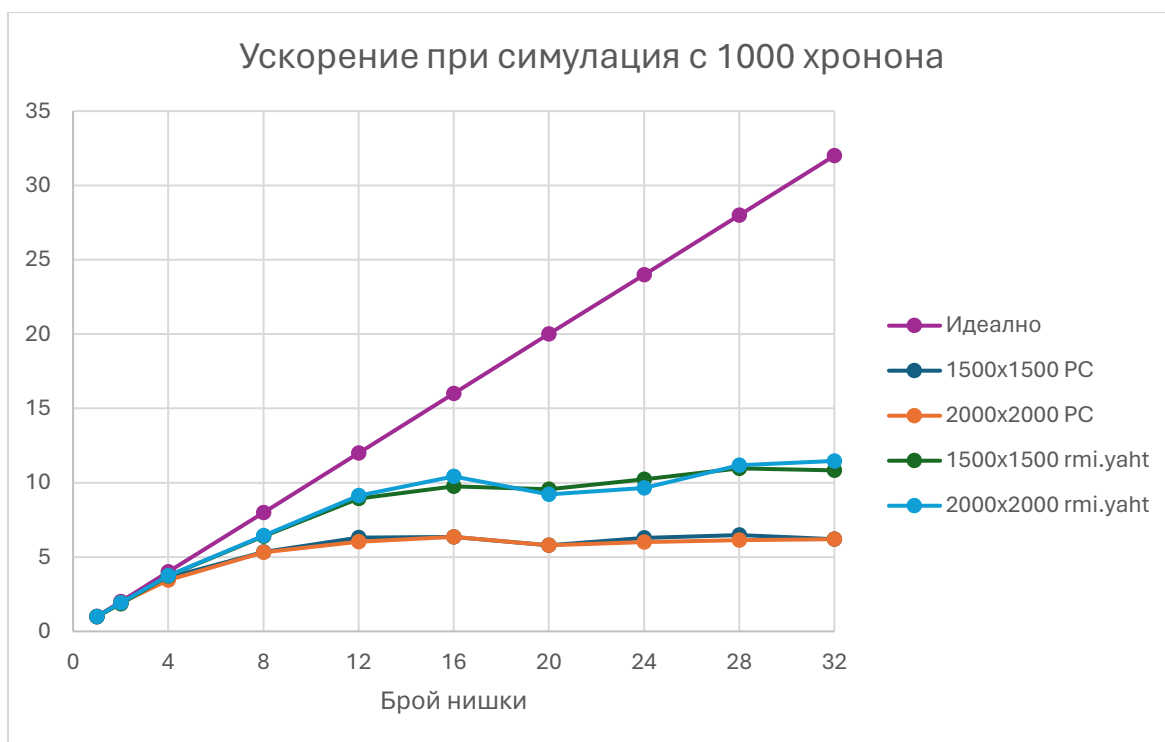
Ускорението и ефективността на паралелизма са представени нагледно на следните графики:



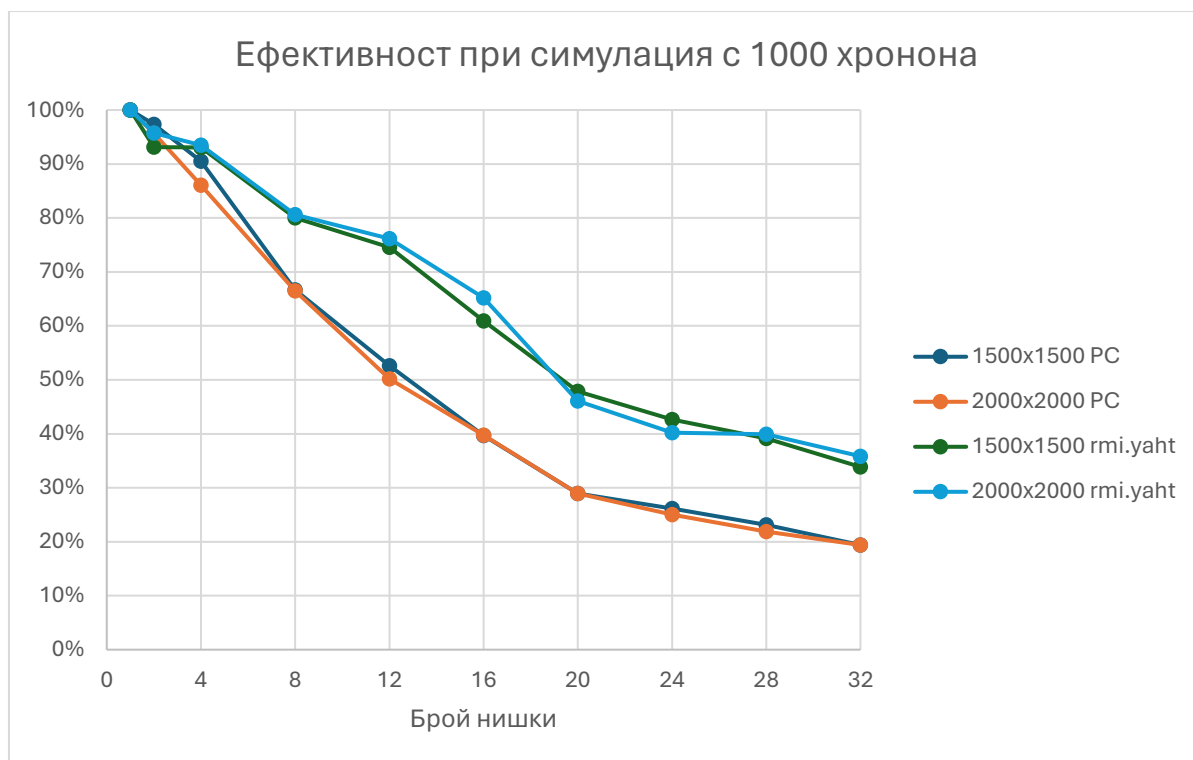
ФИГУРА 10: УСКОРЕНИЕ ПРИ СИМУЛАЦИЯ С 500 ХРОНОНА



ФИГУРА 11: ЕФЕКТИВНОСТ ПРИ СИМУЛАЦИЯ С 500 ХРОНОНА



ФИГУРА 12: УСКОРЕНИЕ ПРИ СИМУЛАЦИЯ С 1000 ХРОНОНА



ФИГУРА 13: ЕФЕКТИВНОСТ ПРИ СИМУЛАЦИЯ С 1000 ХРОНОНА

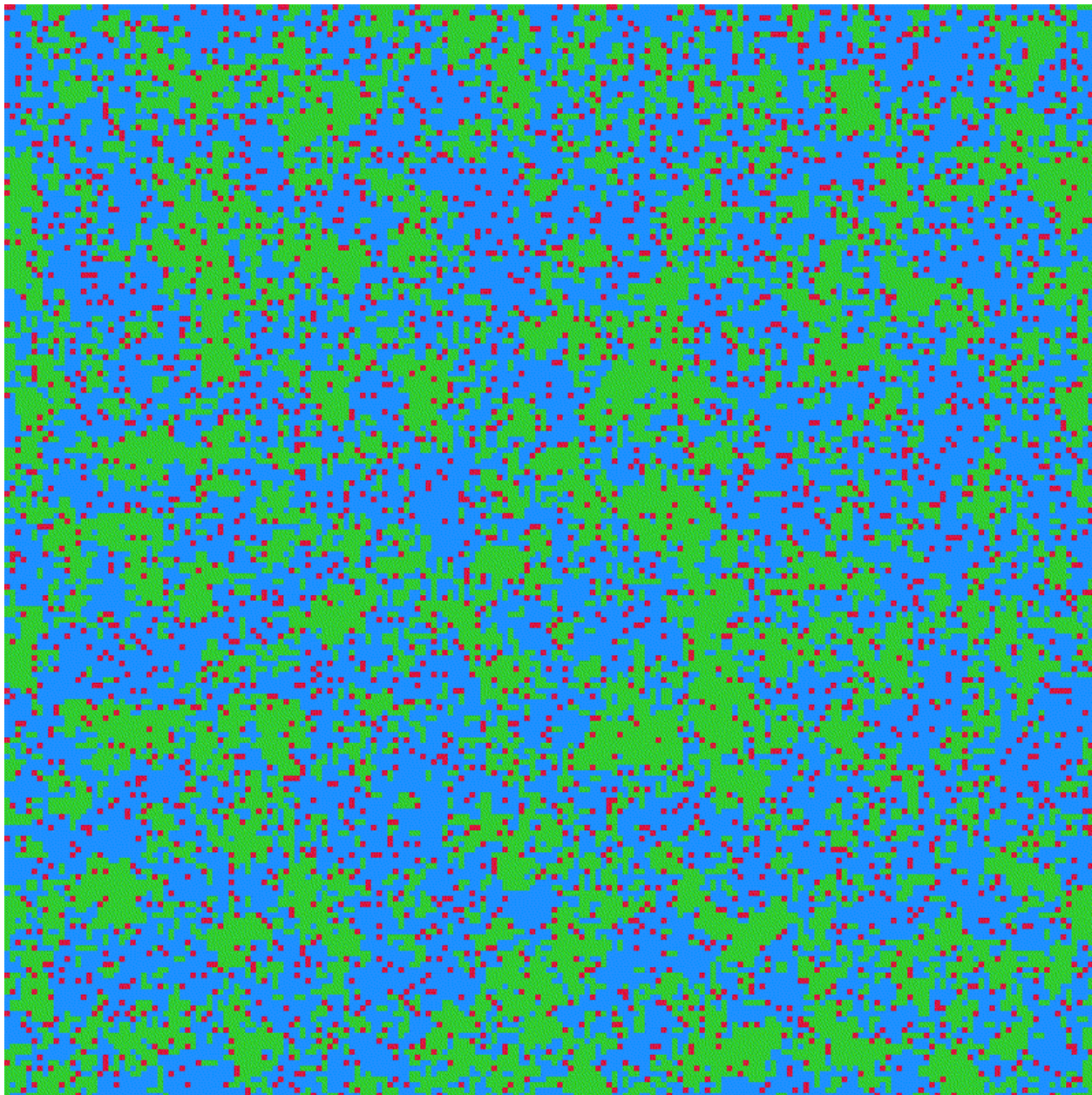
Анализираните резултати потвърждават ефективността на локално-синхронния модел. Прави впечатление, че макар сървърът `rmi.yaht.net` да регистрира по-бавни абсолютни времена за изпълнение (поради по-ниската тактова честота на отделните ядра в сравнение с персоналния компютър), той демонстрира значително по-висока паралелна ефективност при голям брой нишки. Докато по-бързият домашен компютър изчерпва физическия си ресурс и достига плато на ускорението още при 6–8 нишки, сървърът успешно разпределя натоварването и продължава да скалира ефективно до 16 нишки и показва, че способността за мащабиране е много по-важна за крайния успех на симулацията от суровата производителност на единично ядро.

Визуализация

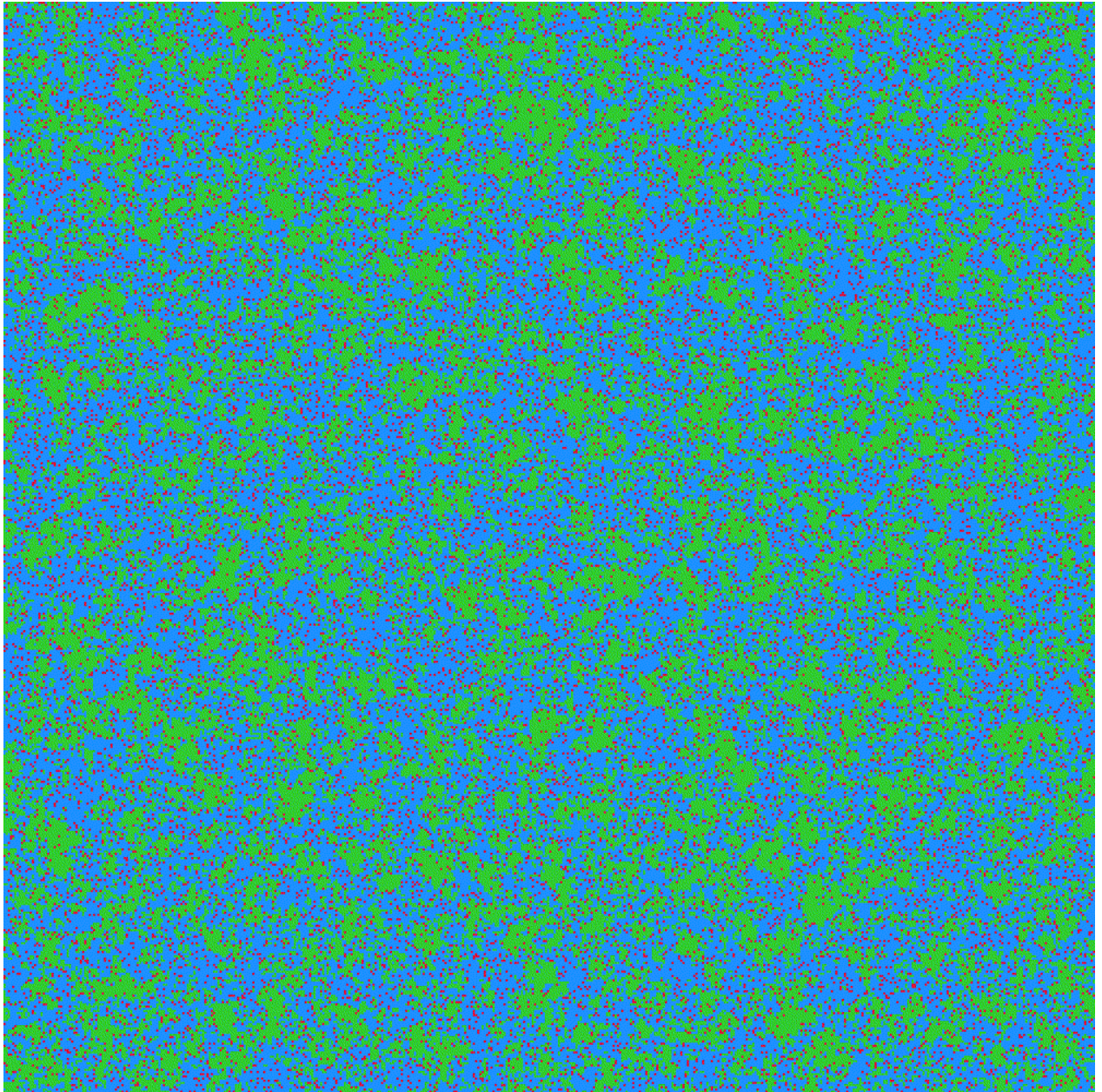
При стартиране на програмата потребителят има избор да включи визуализация на симулацията. Поради забавянето, което тази опция би предизвикала, тя не е използвана по време на тестването. При включена визуализация, по време на изчисленията, на всеки 10 хронона се запазва текущото състояние на матрицата на света под формата на картина „png“ формат. Всяка клетка от матрицата представлява 4x4 пиксела и има цвят, зависещ от типа ѝ:

- **Вода:** светло син;
- **Риба:** зелен;
- **Акула:** червен.

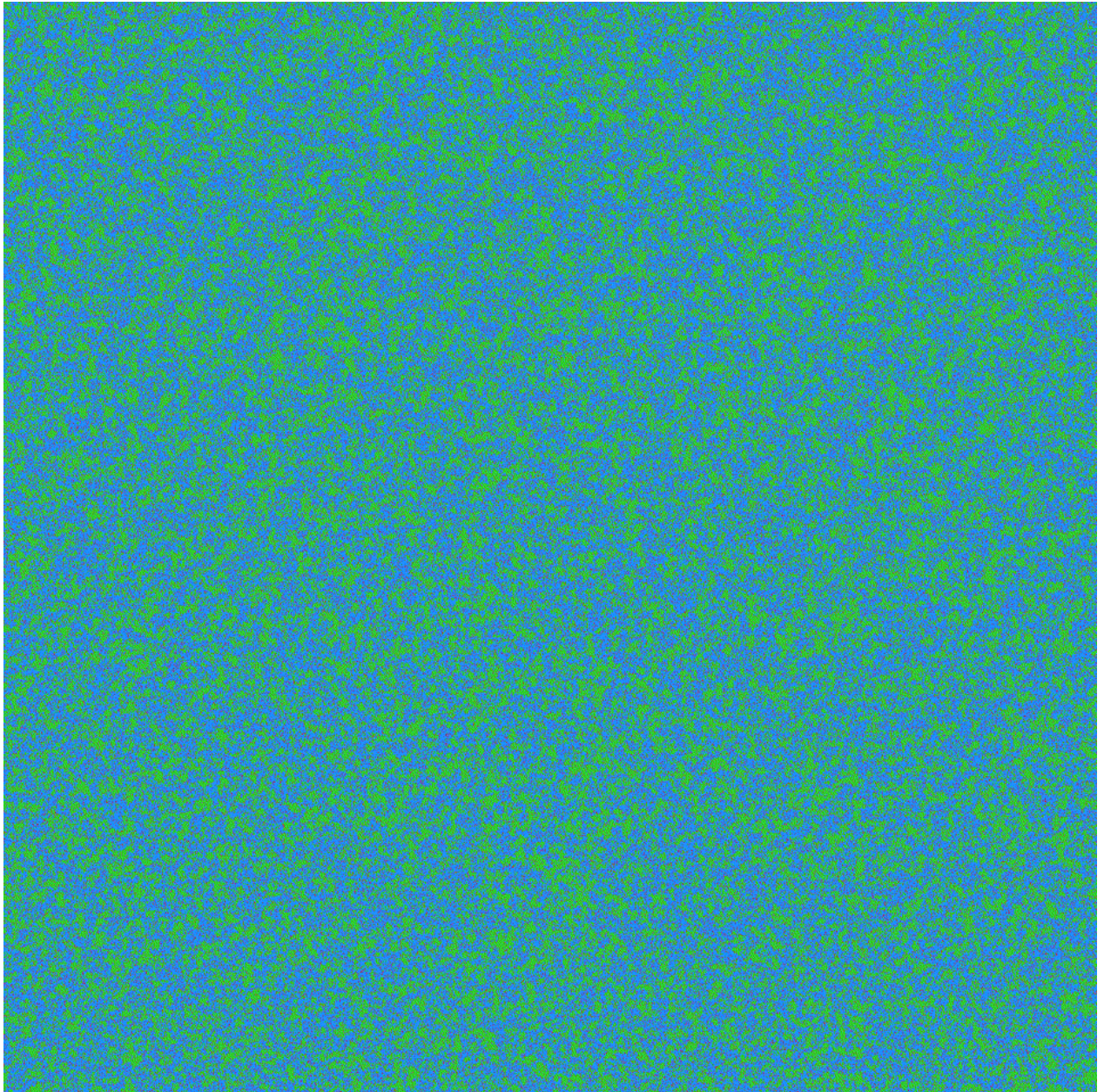
След приключване на изпълнението всички кадри могат да се намерят в новосъздадена папка "frames". Тази папка се изпразва при всяко ново изпълнение, а кадрите в нея могат да се използват от потребителя за създаване на анимация във формат „gif“, илюстрираща популационната динамика в действие. Следващите примерни анимации са със скорост 10 кадъра/сек. и са създадени чрез инструмента **FFmpeg**:



ФИГУРА 14: СИМУЛАЦИЯ 200x200 ЗА 1000 ХРОНОНА



ФИГУРА 15: Симулация 500x500 за 2000 хронона



ФИГУРА 16: Симуляция 1500x1500 за 500 хронона

Заклучение

В заключение, настоящият проект успешно демонстрира проектирането и имплементацията на локално-синхронен паралелен алгоритъм за симулиране на клетъчния автомат Wa-Tor. Чрез ефективна декомпозиция на областта, използване на циклична бариера за времева синхронизация на нишките и система на заключване по границите, беше постигната консистентност на данните. Проведените експерименти потвърдиха теоретичните постановки на Закона на Амдал, доказвайки предимствата на скалируемата архитектура спрямо суровата производителност при голям брой нишки. Като основна възможност за бъдещо развитие и оптимизация на системата може да се разгледа имплементацията на по-усъвършенствано управление на паметта. Тъй като непрекъснатото раждане и умиране на същности в мрежата създава сериозно натоварване върху системата за почистване на паметта, преминаването към стратегии с нулево заделяне на памет по време на симулация (като например използване на матрици от примитивни типове данни) би премахнало това забавяне, увеличавайки производителността при масивни размери на света.

Използвани източници

- [1] A. K. Dewdney, „Sharks and fish Wage an ecological War on the toroidal planet Wa-Tor“, *Scientific American*, pp. 14-22, 1984, (https://github.com/saidone75/wa-tor/blob/main/wator_dewdney.pdf).
- [2] A. Rizzo, „wator-sim“, 2026, (<https://github.com/AgostinoRizzo/wator-sim>).
- [3] N. Tonci, „ParallelWator“, 2016, (<https://github.com/nicolotonci/ParallelWator>).